

Guia: editar e adicionar rotinas na API (YottaDB)

Como funciona a API do moadir, como editar uma rotina existente passo a passo, como criar uma nova e expô-la como endpoint, e o que já se sabe sobre o modelo de dados do ERP legado.

1. Visão geral

A API (porta 9080) é servida pelo YDB Web Server do YottaDB, rodando como o serviço `systemd ydbweb.service`. As rotinas MUMPS ficam em `/root/routines` na VPS.

Três peças formam um endpoint:

Peça	Onde	Papel
Tabela de rotas	<code>/root/routines/_ydbweburl.m</code> (label URLMAP)	mapeia MÉTODO /caminho → label^ROTINA
Rotina adaptadora	<code>/root/routines/API*.m</code> (ex.: <code>APICLI</code> , <code>APIPRD</code>)	lê parâmetros, busca os dados, monta o JSON
Globais de dados	<code>^FCL</code> (clientes), <code>^EPR</code> (produtos), <code>^EEN</code> (encomendas)...	os dados do ERP

Dentro de uma rotina adaptadora, o web server fornece:

- `httpargs("nome")` → parâmetro da query string (`?nome=...`). As chaves chegam sempre em minúsculo, mesmo que o cliente mande `camelCase` na URL (ex.: `?produtoId=5` chega como `httpargs("produtoid")`).
- `httprsp(1)` → corpo da resposta
- `httprsp("mime")` → content-type (use `"text/plain; charset=utf-8"`)

A rotina `ALL.m` tem a legenda do que cada rotina do ERP legado faz (ex.: `E51` = Lista Produtos, `E14` = Edita Produtos, `FAT` = Faturamento). Útil para descobrir onde está a lógica/global original de um assunto antes de replicar como endpoint novo.

Acesso à VPS

```
ssh Moadir # = root@2.25.175.240
```

Endpoints que já existem hoje

Todos em `GET`, com parâmetros na querystring (inclusive os de escrita — ver seção 2, regra de ouro sobre o padrão do projeto).

Rota	Rotina	O que faz
<code>/api/clientes</code>	<code>list^APICLI</code>	Lista clientes
<code>/api/cliente</code>	<code>buscar^APICLI</code>	Busca 1 cliente por id
<code>/api/produtos</code>	<code>list^APIPRD</code>	Lista produtos (matéria-prima/hora-máquina)
<code>/api/produto</code>	<code>buscar^APIPRD</code>	Busca 1 produto por id
<code>/api/produto-criar</code>	<code>criar^APIPRD</code>	Cria produto novo
<code>/api/produto-alterar</code>	<code>alterar^APIPRD</code>	Edita produto existente
<code>/api/encomendas</code>	<code>list^APIEEN</code>	Lista encomendas
<code>/api/encomenda-detalle</code>	<code>detalhe^APIEEN</code>	Detalhe de 1 encomenda (+ movimentos ^GSF)
<code>/api/encomenda-estado</code>	<code>atuest^APIEEN</code>	Atualiza estágio da encomenda
<code>/api/encomenda-criar</code>	<code>criar^APIEEN</code>	Cria encomenda nova
<code>/api/encomenda-alterar</code>	<code>alterar^APIEEN</code>	Edita encomenda existente
<code>/api/encomenda-excluir</code>	<code>excluir^APIEEN</code>	Exclui encomenda (só se ainda sem movimentos)
<code>/api/guia-servico</code>	<code>buscar^APIGSM</code>	Busca a Guia de Serviço (composição) de uma encomenda
<code>/api/guia-servico-salvar</code>	<code>salvar^APIGSM</code>	Grava a composição da Guia de Serviço
<code>/api/consumo-lancar</code>	<code>lançar^APICONS</code>	Lança consumo de material/hora-máquina numa encomenda

Rota	Rotina	O que faz
/api/planilha	planilha^APIPLANILHA	Planilha de Custo completa de uma encomenda
/api/dcp	list^APIDCP	Lista planilhas DCP (prejuízo)
/api/prejuizo	list^APIRELPREJ	Relatório de prejuízo
/api/relatorio , /relatorio	rj^APIRELHTML / rel^APIRELHTML	Relatório em HTML
/api/fatores	list^APIFAT	Lista fatores semanais de correção
/api/fator-set	set^APIFAT	Define/atualiza o fator da semana
/api/debug	debug^APIDEBUG	Diagnóstico da API

2. Regras de ouro (leia antes de mexer)

1. Auto-relink está DESLIGADO. Depois de alterar qualquer rotina você tem que (a) recompilar e (b) reiniciar o `ydbweb` — senão o servidor continua com a versão antiga em memória, mesmo com o `.m` já editado.
2. É produção. O `systemctl restart ydbweb` causa ~3 s de indisponibilidade da API (o dashboard dá erro nesse instante). Faça em horário tranquilo.
3. Sempre faça backup do que vai alterar (`cp arquivo.m arquivo.m.bak`), principalmente o `_ydbweburl.m` (um erro nele derruba todos os endpoints).
4. Compile a rotina sozinha primeiro para pegar erro de sintaxe antes de reiniciar o servidor.
5. Endpoints de escrita também são GET+querystring, não POST com corpo JSON — é o padrão que o projeto já usa em todo lugar (`produto-criar` , `encomenda-alterar` , `guia-servico-salvar` , `consumo-lancar` ...). Siga o mesmo estilo em endpoints novos, mesmo não sendo RESTful "por livro".

3. Passo a passo — editar uma rotina existente

Exemplo prático: ajustar a `APIPRD` (produtos).

1. Conectar e carregar o ambiente

```
ssh Moadir
source /root/ydb-api-env.sh # define $ydb_dist e variáveis do YottaDB
cd /root/routines
```

2. Fazer backup antes de tocar no arquivo

```
cp APIPRD.m APIPRD.m.bak
```

3. Editar

```
nano APIPRD.m
```

(ou `vi` — é texto puro MUMPS, sem ferramenta especial)

4. Compilar sozinha primeiro, pra pegar erro de sintaxe sem afetar quem está usando a API agora:

```
$ydb_dist/mumps APIPRD.m
```

- Sem saída nenhuma = compilou limpo.
- Se der erro, ele aparece na hora — corrija e rode de novo até compilar limpo antes de seguir pro próximo passo.

5. Reiniciar o serviço, pra ele carregar a versão nova em memória:

```
systemctl restart ydbweb.service
```

6. Testar

```
curl -s "http://127.0.0.1:9080/api/produtos?limite=2" # o endpoint que você mexeu
curl -s "http://127.0.0.1:9080/api/clientes?limite=1" # + um endpoint qualquer, garantindo que nada quebrou
```

Se algo der errado, reverta:

```
cp APIPRD.m.bak APIPRD.m
$ydb_dist/mumps APIPRD.m
systemctl restart ydbweb.service
```

Se mexeu também no `_ydbweburl.m` (pra registrar/mudar uma rota), compile os dois arquivos juntos antes de reiniciar — e faça backup dele também, já que um erro ali derruba todos os endpoints de uma vez:

```
cp _ydbweburl.m _ydbweburl.m.bak
$ydb_dist/mumps APIPRD.m _ydbweburl.m
systemctl restart ydbweb.service
```

Se voltar erro 500 / corpo vazio, veja o log do servidor:

```
journalctl -u ydbweb -n 50
```

4. Passo a passo — adicionar uma rotina nova e expor na API

Exemplo fictício: endpoint `GET /api/fornecedores` lendo um global `^F0R`.

Passo 1 — (se necessário) colocar/adaptar a rotina de lógica

Se a lógica vier de outra rotina do legado, copie o `.m` para `/root/routines` e ajuste para MUMPS padrão do YottaDB:

- Nada de ObjectScript/Caché (`##class(...)`, `$zconvert`, `&sql`) nem comandos de GUI proprietária (`ZGW`, `$ZF` ...) — o YottaDB não suporta.
- Confirme que as globais que ela lê (`^XXX`) existem neste banco.
- Compile só para checar sintaxe: `$ydb_dist/mumps MINHAROT.m`.

Passo 2 — criar a rotina adaptadora `API*.m`

Espelhe a `APICLI` / `APIPRD`. Modelo:

```

APIFOR ; API REST - Fornecedores (read-only do ^FOR)
;
list ; GET /api/fornecedores?nome=<filtro>&limite=<n>
NEW filtro,limite,json
SET filtro=$$UPPER($GET(httpargs("nome")))
SET limite=+$GET(httpargs("limite"))
DO BUILDJSON(.json,filtro,limite)
SET httprsp(1)=json
SET httprsp("mime")="text/plain; charset=utf-8"
QUIT
;
BUILDJSON(json,filtro,limite)
NEW id,rec,nome,items,first,n,done
SET items="",first=1,n=0,done=0,id=""
FOR SET id=$ORDER(^FOR(id)) QUIT:id=""!done DO
. SET rec=$GET(^FOR(id))
. SET nome=$PIECE(rec,"^",1)
. QUIT:nome=""
. QUIT:(filtro='')&($FIND($$UPPER(nome),filtro)=0)
. IF 'first SET items=items_"",
. SET first=0
. SET items=items_"{"id":",_+id
. SET items=items_"", "nome":",_,$$CLEAN(nome)_"",
. SET items=items_"", "cidade":",_,$$CLEAN($PIECE(rec,"^",4))_"",
. SET n=n+1
. IF limite>0,n'<limite SET done=1
SET json="{\"total\":",_n_,\"fornecedores\":[_items_]}"
QUIT
;
CLEAN(s) QUIT $TRANSLATE(s,"""_"_"_$CHAR(13)_$CHAR(10))
UPPER(s) QUIT $TRANSLATE(s,"abcdefghijklmnopqrstuvwxyz","ABCDEFGHIJKLMNOPQRSTUVWXYZ")

```

Passo 3 — registrar a rota em _ydbweburl.m

Adicione uma linha no URLMAP, antes do ;;zzzzz:

```
;;GET /api/fornecedores list^APIFOR
```

(O arquivo já tem as rotas de clientes, produtos, encomendas, etc. — veja a tabela completa na seção 1.)

Passo 4 — compilar e reiniciar

```

cp _ydbweburl.m _ydbweburl.m.bak      # backup do roteador!
$ydb_dist/mumps APIFOR.m               # 1. compile a adaptadora sozinha
$ydb_dist/mumps APIFOR.m _ydbweburl.m  # 2. compile as duas
systemctl restart ydbweb.service        # 3. recarrega rotas

```

Passo 5 — testar

```

curl -s "http://127.0.0.1:9080/api/fornecedores?limite=5"
# e confira que os endpoints existentes não quebraram:
curl -s "http://127.0.0.1:9080/api/clientes?limite=1"

```

- 404 → a rota não entrou (revise o _ydbweburl.m e se reiniciou).
- 500 / corpo vazio → erro na rotina (journalctl -u ydbweb -n 50).

Escrevendo endpoints de escrita (criar/alterar)

Sempre que a rotina grava num global (não só lê), valide tudo antes de gravar e devolva {"erro": "..."} cedo se algo estiver errado — nunca grave parcialmente. Padrão usado em APIEEN.m, APIGSM.m, APIPRD.m, APICONS.m:

```
salvar ; GET /api/xxx-salvar?id=&campo=...
NEW id,pr,campo
SET httprsp("mime")="text/plain; charset=utf-8"
SET id=+$GET(httpargs("id"))
IF id<=0 SET httprsp(1)="{\"erro\":\"id invalido\"}" QUIT
SET pr=$GET(^XXX(id))
IF pr="" SET httprsp(1)="{\"erro\":\"nao encontrado\"}" QUIT
;
; ... validar os demais campos, um IF + QUIT por regra ...
;
SET $PIECE(pr,"^",N)=valor ; grava só a piece que a tela edita -
SET ^XXX(id)=pr ; preserva o resto do registro intacto
SET httprsp(1)="{\"ok\":true,\"id\":\"_id_\"}"
QUIT
```

Ao editar um registro existente, prefira `SET $PIECE(rec,"^",N)=valor` em vez de reconstruir a string inteira — assim campos que a tela não expõe (estoque, flags internas, campos legados sem uso conhecido) ficam intocados. Foi assim que `alterar^APIPRD` preserva grupo/estoque do produto ao editar só descrição/custo/venda.

5. Cuidados de MUMPS ao montar JSON

- Sem precedência de operadores — é tudo da esquerda para a direita. `a*1369+b*37` vira `(a*1369+b)*37`. Use parênteses: `(a*1369)+(b*37)`.
- Sempre escape o texto que entra no JSON com a função `CLEAN` (remove aspas, barra e quebras de linha). Strings cruas quebram o `JSON.parse` no app (o app React tem um sanitizador justamente por causa disso).
- Concatenação é com `_` (underscore). Aspas dentro de string M se escrevem duplicando: `""` vira uma aspa.
- Números com decimais: `$JUSTIFY(valor/100,0,2)` formata com 2 casas.
- `$ORDER` percorre subscripts; `$PIECE(rec,"^",n)` pega o campo n (campos separados por `^`). Cuidado com o fim de string: `$ORDER` retorna `""` (string vazia) quando não há mais nada — não um número negativo. Um loop `FOR SET x=$ORDER(...) QUIT:x="" DO ...` é seguro; comparar `x<0` para decidir parar trava o processo num loop infinito, porque `""` em contexto numérico vale `0`, e `0<0` nunca é verdadeiro.
- Arredondamento em MUMPS: `\` é divisão inteira (trunca, não arredonda). Pra arredondar, some meio antes de truncar: `(x+0.5)\1`.

6. Modelo de dados conhecido

Assunto	Global	Rotina API	Observações
Clientes	<code>^FCL(id\168, id\2, id_"1"/"2")</code>	APICLI	<code>/api/clientes</code> , <code>/api/cliente?id=</code>
Produtos	<code>^EPR(k\336, k\4, k)</code>	APIPRD	<code>/api/produtos</code> , <code>/api/produto</code> , <code>/api/produto-criar</code> , <code>/api/produto-alterar</code> . Campos: 1=descrição, 2=unidade, 6/7/8=grupo (3 níveis, uso interno — não exposto na tela), 9=código curto (uso incerto), 10=custo, 11=venda, 12=percentual (uso incerto), 13=local, 14=peso (≠10), 15=origem, 16=classif.fiscal, 17=tributado IPI, 18/19=estoque disponível/reservado. Custo/venda não são o valor em R\$ — são um índice base multiplicado pelo fator da semana vigente (ver abaixo).
Encomendas	<code>^EEN(id\336, id\4, id)</code>	APIEEN	<code>/api/encomendas</code> , <code>/api/encomenda-detalle</code> , <code>-criar</code> , <code>-alterar</code> , <code>-estado</code> , <code>-excluir</code> . Campo 26 = estágio (0=Na Fila...8=Entregue tudo); acima de 5 a encomenda já foi listada na Planilha e não aceita novos lançamentos de consumo.
Guia de Serviço (composição)	<code>^GSM(p1,p2,id)</code>	APIGSM	<code>/api/guia-servico</code> , <code>/api/guia-servico-salvar</code> . Define areceita do pedido (% ou peso de polietileno/mistura/pigmento, sanfona, etc.) — não é consumo real, é o plano de fabricação.
Movimentos de consumo (material/hora-máquina)	<code>^GSF(id\9, id, seq)</code>	APIPLANILHA (leitura) e APICONS (escrita)	<code>/api/planilha</code> , <code>/api/consumo-lancar</code> . Cada movimento: <code>data^tipo^reg^qtdRaw^unidade^produtoCod^auditSeq</code> . <code>tipo</code> : 1=Entrada, 2=Saída/Consumo, 3=Transferência, 4=Devolução/Apara, 5=Ajuste, 6=Cancelamento. <code>produtoCod</code> sempre tem prefixo "P" + id numérico do <code>^EPR</code> — hora-máquina é só um produto comum com unidade "HC", não existe global separado pra mão de obra. Isso é o que alimenta a Planilha de Custo.

Assunto	Global	Rotina API	Observações
Fator semanal	<code>^EIN(AS)</code> (<code>AS=YYWW</code>) e <code>^EIN(2)</code>	<code>APIFAT</code>	<code>/api/fatores</code> , <code>/api/fator-set</code> . <code>^EIN(2)</code> guarda o nº de casas do <code>DIF</code> (<code>DIF="100"_"00"*FUT</code> , ex. <code>FUT=2</code> → <code>DIF=10000</code>). Fórmula de conversão índice-base → R\$ de hoje: <code>reais = (indiceBase * FAT \ DIF) / 100</code> . Usado em todo lugar que envolve preço (produtos, planilha, consumo).
Prejuízo (DCP)	<code>^DCP</code>	<code>APIDCP</code> , <code>APIRELPREJ</code>	<code>/api/dcp</code> , <code>/api/prejuizo</code>

7. Mostrar no app (frontend)

Depois que o endpoint existir, o lado React é:

- `app/lib/api.ts` — tipo + função (`listarX` / `obterX` / `criarX` / `alterarX`) que faz `getJSON` (leitura) da URL nova, passando parâmetros como `queryString` mesmo em escritas.
- `app/routes/x.tsx` — página com `loader` (+ `action` , se for tela de formulário) chamando a função de `~/lib/api` direto — não crie uma rota `api/x.ts` de proxy a menos que precise chamar do client-side via `useFetcher` (ex.: buscas ao vivo tipo `ProdutoPicker`).
- `app/routes.ts` — registrar a rota.
- `app/components/app-menubar.tsx` — item de menu, dentro do grupo (`Cadastros` / `Produção` / `Relatórios`) que fizer mais sentido (opcional).

Veja `docs/` e as páginas existentes (`clientes.tsx` , `produtos.tsx` , `produtos.novo.tsx` , `encomendas.$id.consumo.tsx`) como modelo.

8. Checklist rápido

Editar rotina existente:

```
[ ] ssh Moadir && source /root/ydb-api-env.sh && cd /root/routines
[ ] cp ROTINA.m ROTINA.m.bak
[ ] editar (nano/vi)
[ ] $ydb_dist/mumps ROTINA.m (compila sozinha, sem saída = ok)
[ ] systemctl restart ydbweb.service (em horário tranquilo)
[ ] curl o endpoint mexido + um endpoint qualquer existente
[ ] se quebrou: cp ROTINA.m.bak ROTINA.m && $ydb_dist/mumps ROTINA.m && systemctl restart ydbweb.service
```

Adicionar rotina/endpoint novo:

```
[ ] backup das rotinas que vou mexer (cp X.m X.m.bak) — inclusive _ydbweburl.m
[ ] criar/editar a rotina API*.m (escapar JSON; cuidado com L->R do M; $ORDER termina em "", não <0)
[ ] registrar a rota em _ydbweburl.m (antes do ;;zzzzz)
[ ] $ydb_dist/mumps APIX.m (compila a adaptadora sozinha)
[ ] $ydb_dist/mumps APIX.m _ydbweburl.m
[ ] systemctl restart ydbweb.service (em horário tranquilo)
[ ] curl o endpoint novo + um existente p/ garantir que nada quebrou
[ ] se quebrou: restaurar .bak, recompilar, reiniciar
```